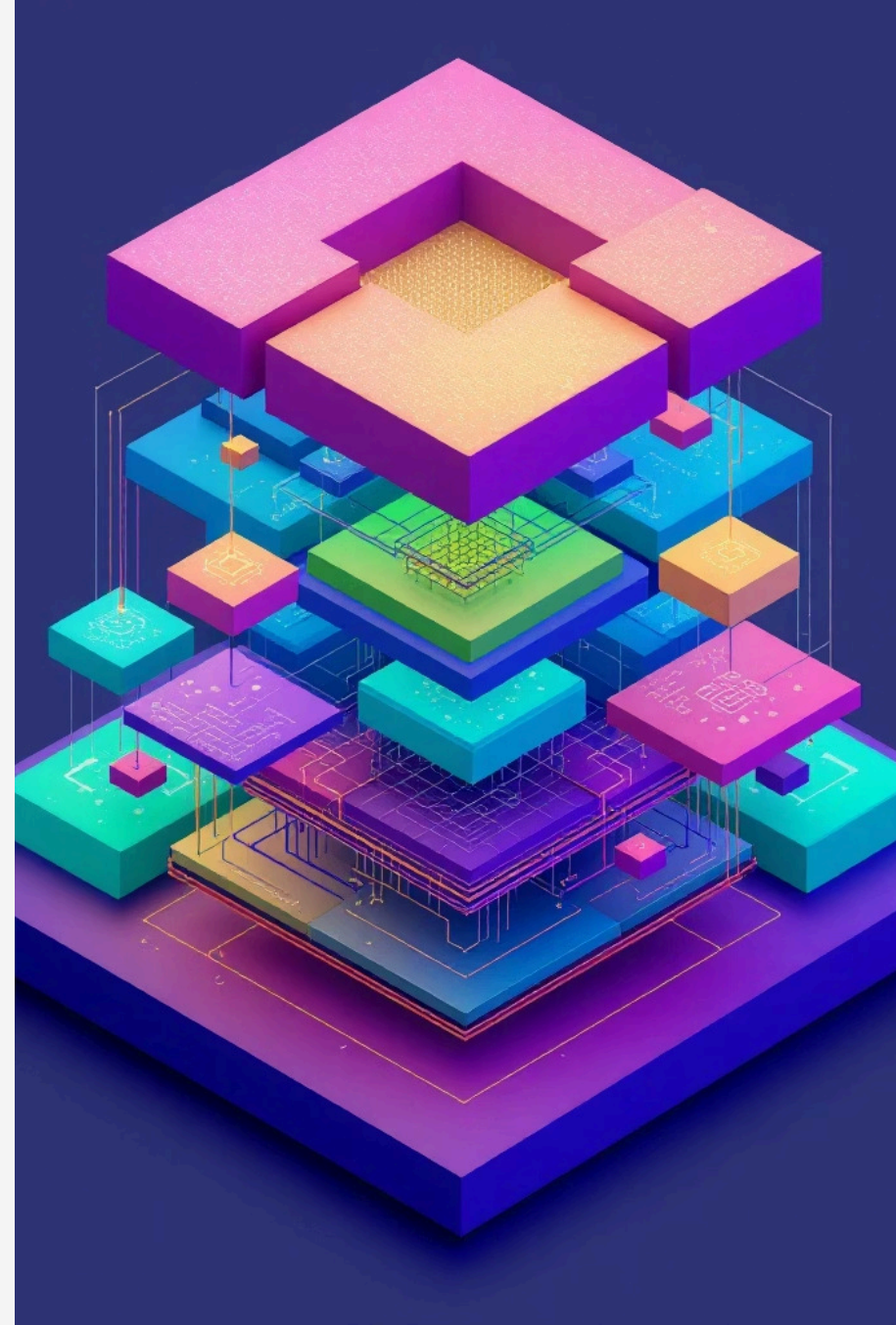


Современные архитектуры CNN

Обзор ключевых архитектур компьютерного зрения, их особенностей и применений.

и by Иван Копылов



Требования к современным архитектурам



Извлечение высокоуровневых семантических признаков

Модель должна выделять контуры, текстуры, формы объектов и объединять их в понятия.



Стабильность градиентного потока

Предотвращение vanishing/exploding gradients при сотнях слоёв.



Эффективная обучаемость

Устойчивость к шуму, достаточная пропускная способность, эффективное масштабирование.



Приемлемая вычислительная сложность

Оптимизация FLOPs, размера модели и времени инференса.

Критерии сравнения архитектур

Метрика	Что измеряет
Top-1 / Top-5 accuracy	Доля корректных предсказаний на ImageNet, CIFAR и др.
mAP	Средняя точность-полнота по классам для детектирования.
FLOPs	Количество операций умножения-сложения на один forward-pass.
Параметры (млн)	Объём весов в модели (в памяти).
Время обучения / инференса	На GPU/CPU, влияет на гибкость исследований и deploy.

Обзор основных приёмов

Малые унифицированные ядра

VGG: использование двух последовательных (3×3) вместо одного (5×5) . Даёт два ReLU вместо одной, меньше параметров.

Residual connections

ResNet: добавление тождественного соединения ($y = F(x) + x$). Обеспечивает единичные собственные значения якобиана.

Многопутевые блоки

Inception: параллельные ветви (1×1) , (3×3) , (5×5) , пулинг. Позволяет модели выбирать «масштаб» автоматически.

Сепарабельные свёртки

MobileNet: depthwise + pointwise. Существенное снижение FLOPs, параметров, сохраняет точность.

Затухание и взрыв градиентов

При обучении глубоких сетей градиент ошибки по входу слоя (l) выражается через цепное правило:

$$\partial L / \partial x^{(l)} = \partial L / \partial x^{(L)} \prod_{k=l}^{L-1} W^{(k)} \text{diag}(\varphi'(z^{(k)}))$$



Проблема

Градиенты либо исчезают, либо взрываются при большой глубине сети.



Методы борьбы

Специальная инициализация (Xavier, He), нормализация слоёв, скип-соединения.



Результат

Стабильное обучение глубоких сетей со сотнями слоёв.

Переобучение и избыточная сложность



Dropout

Случайное "обнуление" нейронов с вероятностью p : $\tilde{x}_i = m_i \cdot x_i / (1-p)$, $m_i \sim \text{Bernoulli}(1-p)$.



Весовой распад

L2-регуляризация: $L' = L + \lambda \sum (W_{ij})^2$. При SGD: $W \leftarrow W - \eta(\nabla_W L + 2\lambda W)$.



Data Augmentation

Рандомные трансформации, Mixup и CutMix (смешивание изображений и меток).



BatchNorm как регуляризатор

Шум батчей и статистика внутри BatchNorm оказывают "мягкий" регуляризующий эффект.

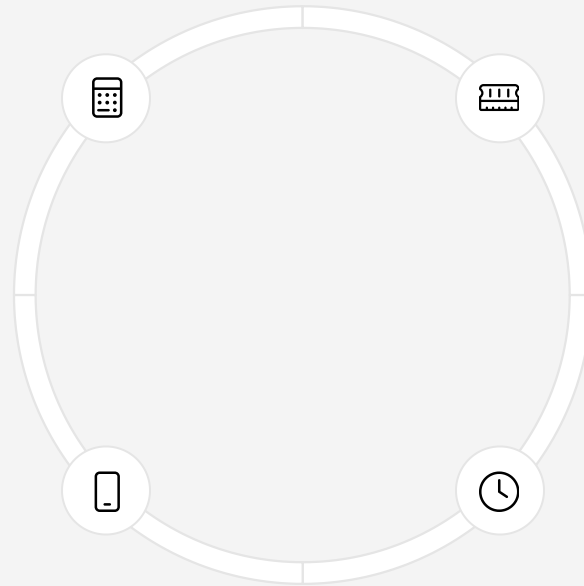
Вычислительные ограничения

FLOPs

Ограничение на количество операций умножения-сложения (до 300M FLOPs).

Энергопотребление

Критично для мобильных и встраиваемых устройств.



Память

Объём весов и активаций (ограничение в десятки мегабайт).

Latency

Задержка не должна превышать 30 ms для работы в реальном времени.

Ключевые архитектурные приёмы

Упрощение и унификация ядра

VGG: две (3×3) вместо одной (5×5) .



Тождественные соединения

ResNet, DenseNet, Inception-ResNet.

Многопутевые блоки

Inception: параллельные ветви разного масштаба.



Сепарабельные свёртки

MobileNet, GhostNet — depthwise + pointwise.

Neural Architecture Search

DARTS, ENAS — автоматический поиск оптимальной структуры.



VGG: простая, но глубокая архитектура

Архитектура VGG (Simonyan & Zisserman, 2014) показала, что простое повторение одинаковых блоков из малых (3×3) свёрток обеспечивает глубокую модель с богатым представлением.



Вход $224\times 224\times 3$

Изображение RGB



Сверточные блоки

Повторяющиеся Conv 3×3 + ReLU + MaxPool

3

Полносвязные слои

FC-4096 $\rightarrow$ FC-4096 $\rightarrow$ FC-1000

Рецептивное поле и параметры VGG

Рецептивное поле

Две последовательные (3×3) свёртки эквивалентны одному (5×5) , но дают две нелинейности:

$$y = \sigma(W_2 * (\sigma(W_1 * x)))$$

Три подряд дают (7×7) .

Количество параметров

Две свёртки (3×3) имеют:

$$2 \times (3^2 C_{in} C_{out}) = 18 \cdot C_{in} \cdot C_{out}$$

Вместо $5^2 C_{in} C_{out} = 25 \cdot C_{in} \cdot C_{out}$ у одного (5×5) фильтра: экономия $\approx 28\%$.

Анализ вычислительных затрат VGG

15.5G

FLOPs

Операций на один forward-pass

138M

Параметры

Общее число весов

120M

FC-слои

Параметры в полносвязных слоях

Первые сверточные блоки (особенно последняя группа 512×3) создают большую часть FLOPs. Плотные слои (FC) имеют наибольшее число параметров, но мало FLOPs по сравнению с conv-частью.

Инициализация и стабильность VGG

Инициализация по Ксавьеру (Glorot)

$$W_{ij} \sim N(0, 2/(n_{in}+n_{out}))$$

Сохраняет дисперсию градиентов на входе и выходе слоя.

BatchNorm (VGG-bn)

Добавление BatchNorm после каждой свёртки:

```
nn.Conv2d(...), nn.BatchNorm2d(...),  
nn.ReLU(inplace=True)
```

Преимущества BatchNorm

Ускоряет сходимость, позволяет брать больший learning rate, уменьшает "плавающие" градиенты.

Пример реализации VGG на PyTorch

```
class VGGBlock(nn.Module):
    def __init__(self, in_ch, out_ch, num_convs):
        super().__init__()
        layers = []
        for _ in range(num_convs):
            layers += [
                nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1),
                nn.BatchNorm2d(out_ch), # для варианта VGG-bn
                nn.ReLU(inplace=True)
            ]
            in_ch = out_ch
        layers += [nn.MaxPool2d(kernel_size=2, stride=2)]
        self.block = nn.Sequential(*layers)

    def forward(self, x):
        return self.block(x)

class VGG16(nn.Module):
    def __init__(self, num_classes=1000):
        super().__init__()
        self.features = nn.Sequential(
            VGGBlock(3, 64, 2),
            VGGBlock(64, 128, 2),
            VGGBlock(128, 256, 3),
            VGGBlock(256, 512, 3),
            VGGBlock(512, 512, 3),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(512*7*7, 4096),
            nn.ReLU(True),
            nn.Dropout(0.5),
            nn.Linear(4096, 4096),
            nn.ReLU(True),
            nn.Dropout(0.5),
            nn.Linear(4096, num_classes),
        )

    def forward(self, x):
        x = self.features(x)
        return self.classifier(x)
```

Преимущества и ограничения VGG

Преимущества

- Архитектурная простота и единообразие блоков
- Глубина 16–19 слоёв обеспечивает хорошую "ёмкость"
- Широкое применение в transfer learning

Ограничения

- Большое число параметров ($\approx 138\text{M}$) и FLOPs ($\approx 15.5\text{G}$)
- Медленное обучение без BatchNorm
- Высокие требования к памяти
- Полносвязные слои съедают значительный объём весов

ResNet: residual learning для сверхглубоких сетей

При наращивании глубины классических CNN (50–100+ слоёв) обучение становится затруднительным из-за затухания градиентов. Чтобы облегчить оптимизацию, в ResNet введены тождественные соединения:

$$y = F(x) + x$$

где $F(x)$ — функция "остатка", реализуемая свёрточными слоями.

Residual-блок

Базовый residual-блок

Состоит из двух (3×3) свёрток:

$$u = \sigma(\text{BN}(W_1 * x))$$

$$v = \text{BN}(W_2 * u)$$

$$y = \sigma(v + x)$$

Согласование размерностей

Если размерности различаются, к (x) применяется (1×1) свёртка (W_s):

$$y = \sigma(W_s x + F(x))$$

Преимущество

Обеспечивает прямой путь для градиента, предотвращая его затухание.

Bottleneck-блок для ResNet-50+



1×1 Conv: $C \rightarrow C/4$

Сжатие каналов



3×3 Conv: $C/4 \rightarrow C/4$

Фильтрация признаков



1×1 Conv: $C/4 \rightarrow C$

Расширение каналов

Собирается так:

$$y = x + W_3 * \sigma(\text{BN}(W_2 * \sigma(\text{BN}(W_1 * x))))$$

Это позволяет глубине расти без квадратичного роста параметров.

Pre-activation ResNet (v2)

Классический ResNet

$$\begin{aligned}u &= \sigma(\text{BN}(W_1 * x)) \\v &= \text{BN}(W_2 * u) \\y &= \sigma(v + x)\end{aligned}$$

Активация после сложения с входом.

Pre-activation ResNet

$$\begin{aligned}\hat{x} &= \sigma(\text{BN}(x)) \\y &= x + W_2 * \sigma(\text{BN}(W_1 * \hat{x}))\end{aligned}$$

Нормализация и активация перед свёртками.

Такой порядок ещё лучше сохраняет поток градиентов и улучшает обобщающую способность.

Анализ градиентов в ResNet

Якобиан residual-блока

$$J = I + \partial F / \partial x$$

Где I - единичная матрица, $\partial F / \partial x$ - якобиан функции остатка.

Собственные значения

$$\lambda_i(J) = 1 + \lambda_i(\partial F / \partial x)$$

Поскольку $\partial F / \partial x$ обычно мал по модулю, $\lambda_i(J) \approx 1$.

Результат

Предотвращение экспоненциального затухания или взрыва градиентов при большой глубине сети.

Пример кода ResNet на PyTorch

```
class ResBlock(nn.Module):
    def __init__(self, C, stride=1, downsample=None):
        super().__init__()
        self.conv1 = nn.Conv2d(C, C, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(C)
        self.relu = nn.ReLU(inplace=True)
        self.conv2 = nn.Conv2d(C, C, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(C)
        self.downsample = downsample

    def forward(self, x):
        identity = x
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        if self.downsample:
            identity = self.downsample(x)

        out += identity
        return self.relu(out)
```

`downsample` — опциональный `nn.Sequential` из (1×1) Conv и BatchNorm для согласования размерностей при изменении числа каналов или `stride`.

Плюсы и минусы ResNet

Плюсы

- Позволяет строить сверхглубокие сети (50–152 слоёв)
- Быстрая и устойчивая конвергенция
- Улучшенная обобщающая способность

Минусы

- Усложнение реализации блоков
- Контроль размерностей при переходе между группами слоёв
- Эффект "diminishing returns" при чрезмерной глубине

Inception-ResNet модули

Inception-ResNet комбинирует идеи многопутевого анализа (Inception) и residual-соединений (ResNet). Для стабильности применяется масштабирование "остатка" малым коэффициентом ($\alpha \approx 0.1$).

Inception-ResNet-A модуль



Ветвь 1

(1×1) Conv $\rightarrow$ ReLU, выход 32 каналов



Ветвь 2

(1×1) Conv $\rightarrow$ ReLU $\rightarrow (3 \times 3)$ Conv, выход 32 канала



Ветвь 3

$(1 \times 1) \rightarrow (3 \times 3) \rightarrow (3 \times 3)$ Conv с ReLU, выход 64 канала



Ветвь 4

(3×3) AvgPool $\rightarrow (1 \times 1)$ Conv, выход 64 канала

Все выходы конкатенируются (192 канала), применяется проекционный (1×1) Conv, затем прибавляется вход, умноженный на $\alpha=0.1$.

Inception-ResNet-B модуль

1

Ветвь 1

(1×1) Conv $\rightarrow$ ReLU, выход 192 канала

2

Ветвь 2

$(1 \times 1) \rightarrow (1 \times 7) \rightarrow (7 \times 1)$ Conv с ReLU, выход 192 канала

3

Ветвь 3

(3×3) AvgPool $\rightarrow (1 \times 1)$ Conv, выход 192 канала

Конкатенация даёт 576 каналов, далее проекция (1×1) и residual-скейл с $\alpha=0.1$.

Применение модулей A и B

Модули A

Применяются в начальной части сети, где ширина тензора (число каналов) невелика (256–384).

Используют стандартные квадратные свёртки 3×3 .

Модули B

Более "тяжёлые" по ветви $(1 \times 7) + (7 \times 1)$, используются после уменьшения пространственного размера.

Применяются при увеличении каналов (768–1024).

Пример кода Inception-ResNet-A

```
class InceptionResNetA(nn.Module):
    def __init__(self, in_ch, scale=0.1):
        super().__init__()
        self.scale = scale

        # Ветвь 1
        self.b1 = nn.Conv2d(in_ch, 32, 1)

        # Ветвь 2
        self.b2 = nn.Sequential(
            nn.Conv2d(in_ch, 32, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(32, 32, 3, padding=1)
        )

        # Ветвь 3
        self.b3 = nn.Sequential(
            nn.Conv2d(in_ch, 48, 1),
            nn.ReLU(inplace=True),
            nn.Conv2d(48, 64, 3, padding=1),
            nn.ReLU(inplace=True),
            nn.Conv2d(64, 64, 3, padding=1)
        )

        # Ветвь 4
        self.b4 = nn.Sequential(
            nn.AvgPool2d(3, stride=1, padding=1),
            nn.Conv2d(in_ch, 64, 1)
        )

        # Проекция
        self.proj = nn.Conv2d(192, in_ch, 1)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        b1 = self.b1(x)
        b2 = self.b2(x)
        b3 = self.b3(x)
        b4 = self.b4(x)

        out = torch.cat([b1, b2, b3, b4], dim=1)
        out = self.proj(out).mul(self.scale)

        return self.relu(x + out)
```

Роль коэффициента α

Проблема

При объединении параллельных ветвей могут возникать резкие скачки активаций.

Решение

Умножение "остатка" на $\alpha \approx 0.1$ стабилизирует обучение очень глубоких сетей.

Эффект

Предотвращает взрывной рост активаций и градиентов при прохождении через множество слоёв.

DenseNet: плотная конкатенация

В DenseNet (Huang et al., 2017) предложена плотная схема соединений: каждый слой получает конкатенацию выходов всех предыдущих слоёв в том же блоке:

$$x_l = H_l([x_0, x_1, \dots, x_{l-1}])$$

где $H_l(\cdot)$ обычно реализуется как последовательность BatchNorm $\rightarrow$ ReLU $\rightarrow$ Conv3×3.

Преимущества DenseNet



Переиспользование признаков

Поздние слои могут напрямую обратиться к "ранним" низкоуровневым признакам.



Экономия параметров

Новые слои добавляют лишь небольшое число новых каналов.



Упрощённый градиентный поток

Каждый слой имеет прямой путь до функции потерь.



Implicit deep supervision

Не требуется добавлять auxiliary losses, градиент "видит" ранние слои напрямую.

Темп прироста (growth rate)

Каждый слой в Dense-блоке добавляет фиксированное число (k) новых каналов — growth rate. Если вход блока имеет (C_0) каналов, то после (L) слоёв их станет:

$$C_L = C_0 + k \cdot L$$

Типичные значения (k) — 12, 16, 32. Небольшой (k) позволяет строить очень глубокие сети без взрывного роста числа параметров.

Переходные слои между dense-блоками

BatchNorm → ReLU → Conv1×1

Для сжатия числа каналов: $C_{out} = \lfloor \theta \cdot C_{in} \rfloor$, $\theta \in (0,1]$

AvgPool2×2, stride=2

Для снижения пространственных размеров вдвое

Результат

Контроль числа каналов и пространственного разрешения между блоками

Обычно $\theta=0.5$, то есть число каналов сокращается вдвое.

Код DenseLayer и Transition на PyTorch

```
class DenseLayer(nn.Module):
    def __init__(self, in_ch, growth_rate):
        super().__init__()
        self.bn = nn.BatchNorm2d(in_ch)
        self.relu = nn.ReLU(inplace=True)
        self.conv = nn.Conv2d(in_ch, growth_rate, kernel_size=3,
                               padding=1, bias=False)

    def forward(self, x):
        out = self.conv(self.relu(self.bn(x)))
        # Конкатенируем вход и выход
        return torch.cat([x, out], dim=1)

class Transition(nn.Module):
    def __init__(self, in_ch, out_ch):
        super().__init__()
        self.bn = nn.BatchNorm2d(in_ch)
        self.relu = nn.ReLU(inplace=True)
        self.conv = nn.Conv2d(in_ch, out_ch, kernel_size=1, bias=False)
        self.pool = nn.AvgPool2d(kernel_size=2, stride=2)

    def forward(self, x):
        x = self.conv(self.relu(self.bn(x)))
        return self.pool(x)
```

Плюсы и минусы DenseNet

Плюсы

- Эффективное переиспользование признаков
- Ускоренная сходимость
- Умеренный рост параметров

Минусы

- Вырастает число каналов внутри блока
- Затраты на конкатенацию
- Высокие требования к памяти

MobileNet: глубинно-сепарабельные свёртки

Классическая свёртка ($K \times K$) на входе размером ($H \times W \times C_{in}$) с (C_{out}) выходными каналами требует:

$$\text{Cost}_{std} = K^2 \cdot C_{in} \cdot C_{out} \cdot H \cdot W$$

В depthwise-separable свёртке операция разбивается на два этапа:

Depthwise Separable Convolution

Depthwise

(K×K) свёртка по каждому входному каналу отдельно:

$$\text{Cost}_{\text{dw}} = K^2 \cdot C_{\text{in}} \cdot H \cdot W$$

Pointwise

(1×1) Conv для смешивания каналов:

$$\text{Cost}_{\text{pw}} = C_{\text{in}} \cdot C_{\text{out}} \cdot H \cdot W$$

Итого

$$\text{Cost}_{\text{sep}} \approx (1/C_{\text{out}} + 1/K^2) \times \text{Cost}_{\text{std}}$$

При K=3 и $C_{\text{out}} \gg 1$ даёт примерно 8–9× экономию FLOPs.

Width и Resolution Multipliers

Width multiplier (α)

Меняет число каналов:

$$C'_{in} = \alpha \cdot C_{in}$$
$$C'_{out} = \alpha \cdot C_{out}$$

$\alpha \in (0,1]$ - коэффициент сжатия каналов.

Resolution multiplier (ρ)

Меняет пространственный размер:

$$H' = \rho \cdot H$$
$$W' = \rho \cdot W$$

$\rho \in (0,1]$ - коэффициент сжатия разрешения.

FLOPs масштабируются примерно как $\alpha^2 \cdot \rho^2$. Например, $\alpha=0.5$, $\rho=0.75$ даёт $0.5^2 \times 0.75^2 \approx 0.14$ от исходной нагрузки.

MobileNetV2: inverted residuals



Расширение

(1×1) Conv ($C_{in} \rightarrow t \cdot C_{in}$), expand ratio $t=6$

2

Depthwise

(3×3) Conv по каждому из $t \cdot C_{in}$ каналов



Сжатие (linear)

(1×1) Conv ($t \cdot C_{in} \rightarrow C_{out}$) без ReLU

Блок соединяется с входом (skip-connection) только если $stride=1$ и $C_{in}=C_{out}$:

$$y = x + B(x)$$

где B — composed expansion $\rightarrow$ depthwise $\rightarrow$ projection.

Примеры кода MobileNet на PyTorch

```
class DepthwiseSeparable(nn.Module):
    def __init__(self, in_ch, out_ch, stride):
        super().__init__()
        # depthwise
        self.dw = nn.Conv2d(in_ch, in_ch, kernel_size=3,
                             stride=stride, padding=1,
                             groups=in_ch, bias=False)
        self.bn1 = nn.BatchNorm2d(in_ch)

        # pointwise
        self.pw = nn.Conv2d(in_ch, out_ch, kernel_size=1,
                             stride=1, padding=0, bias=False)
        self.bn2 = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU6(inplace=True)

    def forward(self, x):
        x = self.relu(self.bn1(self.dw(x)))
        x = self.relu(self.bn2(self.pw(x)))
        return x
```

Inverted Residual Block (MobileNetV2)

```
class InvertedResidual(nn.Module):
    def __init__(self, in_ch, out_ch, stride, expand_ratio):
        super().__init__()
        self.stride = stride
        mid_ch = in_ch * expand_ratio

        # Определяем, нужен ли skip
        self.use_res_connect = (self.stride == 1 and in_ch == out_ch)

        layers = []
        # 1x1 расширение
        layers.append(nn.Conv2d(in_ch, mid_ch, kernel_size=1, bias=False))
        layers.append(nn.BatchNorm2d(mid_ch))
        layers.append(nn.ReLU6(inplace=True))

        # depthwise
        layers.append(nn.Conv2d(mid_ch, mid_ch, kernel_size=3,
                                stride=stride, padding=1,
                                groups=mid_ch, bias=False))
        layers.append(nn.BatchNorm2d(mid_ch))
        layers.append(nn.ReLU6(inplace=True))

        # 1x1 сжатие (linear)
        layers.append(nn.Conv2d(mid_ch, out_ch, kernel_size=1, bias=False))
        layers.append(nn.BatchNorm2d(out_ch))

        self.conv = nn.Sequential(*layers)

    def forward(self, x):
        if self.use_res_connect:
            return x + self.conv(x)
        else:
            return self.conv(x)
```

Плюсы и ограничения MobileNet

Плюсы

- Существенное снижение FLOPs и параметров
- Гибкость благодаря width/resolution multipliers
- Хорошая точность при очень лёгкой модели

Минусы

- Меньшее ускорение на GPU
- Небольшое падение Top-1 accuracy
- Чувствительность к качеству инициализации

EfficientNet: compound scaling

До появления EfficientNet многие подходы к увеличению мощности сети масштабировали только один из трёх аспектов:



Глубину (depth)

Добавляя больше слоёв (ResNet, VGG)



Ширину (width)

Увеличивая число каналов (Wide ResNet)



Разрешение (resolution)

Подавая на вход более крупные изображения

Такой разрозненный рост часто давал убывающую отдачу.

Compound scaling в EfficientNet

EfficientNet (Tan & Le, 2019) предложил сбалансированный метод масштабирования всех трёх измерений с единым фактором (ϕ):

$$d = \alpha^{\phi}$$

$$w = \beta^{\phi}$$

$$r = \gamma^{\phi}$$

при условии $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$, $\alpha \geq 1$, $\beta \geq 1$, $\gamma \geq 1$.

Это гарантирует, что при увеличении ресурсов ($\text{GFLOPs} \approx 2^{\phi}$) глубина, ширина и разрешение растут сбалансированно.

Базовая архитектура EfficientNet-B0

Ручное проектирование базовой сети B0 сочетает в себе inverted residual blocks (MBConv) с squeeze-and-excitation.

Оператор	Ядро	Повторы	Каналы in→out
Conv2d	3×3	1	3→32
MBConv1	3×3	1	32→16
MBConv6	3×3	2	16→24
MBConv6	5×5	2	24→40
MBConv6	3×3	3	40→80

Пример кода MBConv в PyTorch

```
class SqueezeExcite(nn.Module):
    def __init__(self, in_ch, se_ratio=0.25):
        super().__init__()
        reduced_ch = max(1, int(in_ch * se_ratio))
        self.fc1 = nn.Conv2d(in_ch, reduced_ch, kernel_size=1)
        self.fc2 = nn.Conv2d(reduced_ch, in_ch, kernel_size=1)

    def forward(self, x):
        # Global mean pooling
        se = F.adaptive_avg_pool2d(x, 1)
        se = F.relu(self.fc1(se), inplace=True)
        se = torch.sigmoid(self.fc2(se))
        return x * se

class MBConv(nn.Module):
    def __init__(self, in_ch, out_ch, expand_ratio, kernel_size,
                 stride, se_ratio=0.25):
        super().__init__()
        mid_ch = in_ch * expand_ratio
        self.use_res = (stride == 1 and in_ch == out_ch)

        layers = []
        # Expand
        if expand_ratio != 1:
            layers += [
                nn.Conv2d(in_ch, mid_ch, kernel_size=1, bias=False),
                nn.BatchNorm2d(mid_ch),
                nn.SiLU(inplace=True)
            ]

        # Depthwise
        layers += [
            nn.Conv2d(mid_ch, mid_ch, kernel_size=kernel_size,
                      stride=stride, padding=kernel_size//2,
                      groups=mid_ch, bias=False),
            nn.BatchNorm2d(mid_ch),
            nn.SiLU(inplace=True)
        ]

        # Squeeze-and-Excitation
        layers.append(SqueezeExcite(mid_ch, se_ratio))

        # Project
        layers += [
            nn.Conv2d(mid_ch, out_ch, kernel_size=1, bias=False),
            nn.BatchNorm2d(out_ch)
        ]

        self.block = nn.Sequential(*layers)

    def forward(self, x):
        out = self.block(x)
        return x + out if self.use_res else out
```

Результаты EfficientNet

Модель	Тор-1 (%)	Параметры (М)	FLOPs (G)
EfficientNet-B0	77.1	5.3	0.39
EfficientNet-B1	79.2	7.8	0.70
EfficientNet-B3	81.6	12	1.8
EfficientNet-B5	83.6	30	9.9
EfficientNet-B7	84.3	66	37

B7 достигает 84.3% Тор-1 на ImageNet при 37 GFLOPs и 66М параметров, значительно опережая ResNet-50/101 и Inception-v3 в том же бюджетном диапазоне.

Attention-модули: Squeeze-and-Excitation (SE)

Squeeze

Глобальное среднее объединение по пространственным осям:

$$z_c = 1/(H \cdot W) \sum_{i=1}^H \sum_{j=1}^W X_{\{i,j,c\}}$$

Excitation

Два полносвязных слоя с non-linearity и сигмной:

$$s = \sigma(W_2 \cdot \delta(W_1 \cdot z))$$

Scale

По-элементное умножение карт признаков на их каналные веса:

$$\tilde{X}_{\{i,j,c\}} = s_c \cdot X_{\{i,j,c\}}$$

Convolutional Block Attention Module (CBAM)

Channel Attention

Вычисляем два агрегата по каналам:

$$M_{avg} = 1/(H \cdot W) \sum_{i,j} X_{i,j,:}$$

$$M_{max} = \max_{i,j} X_{i,j,:}$$

Скормить каждый в общий MLP, получить два вектора, сложить и применить σ .

Spatial Attention

По каналам считаем среднее и максимум:

$$A_{avg} = 1/C \sum_c X_c$$

$$A_{max} = \max_c X_c$$

Конкатенируем $[A_{avg}, A_{max}]$ и пропускаем через (7×7) Conv + σ , получая карту M_s .

Преимущества и ограничения Attention-модулей

Плюсы

- Улучшает выразительность за счёт динамического взвешивания
- Лёгко интегрируется в любую CNN
- Небольшой оверхед (пара сотен килобайт весов)

Минусы

- Некоторая задержка из-за дополнительных вычислений
- Гиперпараметры требуют настройки
- Не всегда даёт значительный прирост точности

Neural Architecture Search (NAS)

RL-based NAS

Архитектура генерируется агентом (RNN), который пробует варианты сетей. Примеры: NASNet, ENAS.

Оценка и выбор

Вознаграждение комбинирует качество на validation set и эффективность (FLOPs, latency).



Evolutionary NAS

Эволюционные алгоритмы создают новые поколения архитектур. Примеры: AmoebaNet, Regularized Evolution.

Differentiable NAS

Архитектурные решения отображаются в непрерывные параметры (α). Пример: DARTS.

Сравнительный анализ и заключение

Архитектура	Top-1 (%)	Параметры (М)	FLOPs (G)
VGG-16	71.5	138	15.5
ResNet-50	76.2	26	4.1
Inception-v3	78.8	23	5.7
MobileNet-V2	72.0	3.4	0.3
EfficientNet-B7	84.3	66	37

Ключевые идеи: skip-connections компенсируют затухание градиентов, многопутевые блоки захватывают признаки разных масштабов, сепарабельные свёртки оптимизируют FLOPs/параметры, compound scaling сбалансировано растит глубину, ширину, разрешение.